Universität Konstanz

Introduction to Computing for the Social Sciences

Session 4 – Software

David Garcia

Winter Semester 2026-2027

Course Outline

- *Session 1: Introduction*
- *Session 2: Information Coding*
- *Session 3: Data Structures*
- **Session 4: Software**
- Session 5: Algorithms
- Session 6: Recursion
- Session 7: Sorting Algorithms
- Session 8: Time Complexity
- Session 9: Theoretical Computer Science
- Session 10: Machine Learning
- Session 11: Databases
- Session 12: Parallel Programming
- Session 13: Social Science Applications
- Session 14: Exam Review

Outline

1. Programming in a nutshell
2. Software engineering
3. Requirements, specifications, and design
4. Testing

The app that leaked 13,000 photo IDs

Universität Konstanz

MOTIVATING EXAMPLE

Tea, a US app where women warn each other about men they date, asked every new user for a selfie and a photo ID to prove she was a woman. In July 2025 someone posted the address of the storage contained all these data. There was **no login and no access rule.** Anyone could download everything.

72,000
images downloaded

13,000
selfies and photo IDs

1 million+
private messages, in a second leak

***"Only moderators may see the ID photos"* was obviously required. Why did nothing enforce it, and which test would have caught it?**

TechCrunch and CNN, 26 July 2025; 404 Media. Many blamed vibe coding; Tea says the code predates 2024.

20 million lines of code to move 5 millimeters

Universität Konstanz

MOTIVATING EXAMPLE

In the 2000s, **Symbian**, the system of Nokia's phones, ran most of the world's smartphones. In 2010 a former Nokia design executive told how his team wanted to move one element of a phone's screen up by 5 mm: *"We can't find the piece of code that could shift it up 5mm."*

20 million
lines of code in one phone

40 million
lines of Symbian released as open source, 2010

11 million
lines of Android at the time

Nobody wrote this code to be unchangeable. What makes working code impossible to change? What keeps a codebase changeable?

Juhani Risku in The Register, 22 July 2010; All About Symbian, 4 February 2010. In 2011 Nokia replaced Symbian with Windows Phone.

Programming in a nutshell

Universität Konstanz

1. **Programming in a nutshell**
2. Software engineering
3. Requirements, specifications, and design
4. Testing

What is programming?

Universität Konstanz

Definition: Programming

Programming is the process of creating a set of instructions that tell a computer how to perform a task.

Definition: Programming Language

A formal language for writing those instructions, either directly as machine code or as human-readable source code that is translated to machine code automatically.

Vibe coding is programming too

A coding agent automatically converts natural language into machine code, thus vibe coding is a type of programming. The main difference with traditional coding is **flakiness**: the same natural language request can lead to very different codes as a result.

The first programmer: Ada Lovelace



- **1843**: Ada Lovelace (1815–1852) writes notes on Babbage's **Analytical Engine**
- **Note G** (right): a program for the engine to compute Bernoulli numbers, the first published program

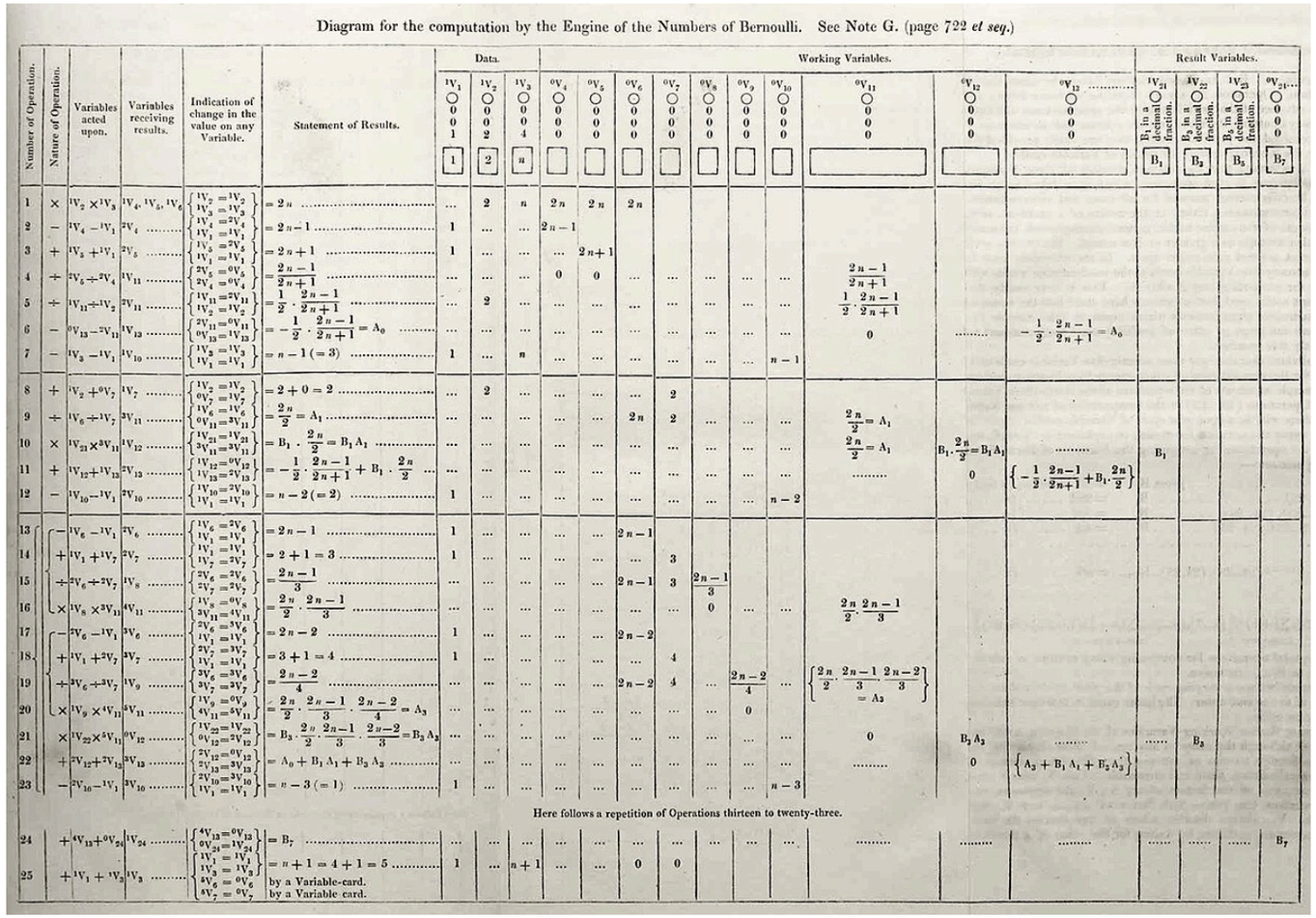


Diagram for the computation by the Engine of the Numbers of Bernoulli. See Note G. (page 722 *et seq.*)

Number of Operation.	Nature of Operation.	Variables acted upon.	Variables receiving results.	Indication of change in the value on any Variable.	Statement of Results.	Data. $^1V_1$ 1	Data. $^1V_2$ 2	Data. $^1V_3$ n	Working Variables. $^0V_4$	$^0V_5$	$^0V_6$	$^0V_7$	$^0V_8$	$^0V_9$	$^0V_{10}$	$^0V_{11}$	$^0V_{12}$	$^0V_{13}$	Result Variables. $^1V_{21}$ B_1	$^1V_{22}$ B_3	$^1V_{23}$ B_5	$^0V_{24}$ B_7
1	×	$^1V_2 \times ^1V_3$	$^1V_4, ^1V_5, ^1V_6$	$^1V_2 = ^1V_2$, $^1V_3 = ^1V_3$	$= 2n$	...	2	n	2n	2n	2n											
2	−	$^1V_4 - ^1V_1$	$^2V_4$	$^1V_4 = ^2V_4$, $^1V_1 = ^1V_1$	$= 2n-1$	1	...	...	2n−1													
3	+	$^1V_5 + ^1V_1$	$^2V_5$	$^1V_5 = ^2V_5$, $^1V_1 = ^1V_1$	$= 2n+1$	1	...	...	...	2n+1												
4	÷	$^2V_5 \div ^2V_4$	$^1V_{11}$	$^2V_5 = ^0V_5$, $^2V_4 = ^0V_4$	$= \frac{2n-1}{2n+1}$	...	...	...	0	0	...	...	...	...	...	$\frac{2n-1}{2n+1}$						
5	÷	$^1V_{11} \div ^1V_2$	$^2V_{11}$	$^1V_{11} = ^2V_{11}$, $^1V_2 = ^1V_2$	$= \frac{1}{2} \cdot \frac{2n-1}{2n+1}$	...	2	...	...	...	...	...	...	...	...	$\frac{1}{2} \cdot \frac{2n-1}{2n+1}$						
6	−	$^0V_{13} - ^2V_{11}$	$^1V_{13}$	$^2V_{11} = ^0V_{11}$, $^0V_{13} = ^1V_{13}$	$= -\frac{1}{2} \cdot \frac{2n-1}{2n+1} = A_0$	...	...	...	...	...	...	...	...	...	...	0	.....	$-\frac{1}{2} \cdot \frac{2n-1}{2n+1} = A_0$					
7	−	$^1V_3 - ^1V_1$	$^1V_{10}$	$^1V_3 = ^1V_3$, $^1V_1 = ^1V_1$	$= n-1\ (=3)$	1	...	n	...	...	...	...	...	...	n − 1							
8	+	$^1V_2 + ^0V_7$	$^1V_7$	$^1V_2 = ^1V_2$, $^0V_7 = ^1V_7$	$= 2 + 0 = 2$	...	2	...	...	...	...	2										
9	÷	$^1V_6 \div ^1V_7$	$^2V_{11}$	$^1V_6 = ^1V_6$, $^0V_{11} = ^2V_{11}$	$= \frac{2n}{2} = A_1$	...	...	...	...	...	2n	2	...	...	...	$\frac{2n}{2} = A_1$						
10	×	$^1V_{21} \times ^2V_{11}$	$^1V_{12}$	$^1V_{21} = ^1V_{21}$, $^2V_{11} = ^2V_{11}$	$= B_1 \cdot \frac{2n}{2} = B_1 A_1$	...	...	...	...	...	...	...	...	...	...	$\frac{2n}{2} = A_1$	$B_1 \cdot \frac{2n}{2} = B_1 A_1$	.....	B_1			
11	+	$^1V_{12} + ^1V_{13}$	$^2V_{13}$	$^1V_{12} = ^0V_{12}$, $^1V_{13} = ^2V_{13}$	$= -\frac{1}{2} \cdot \frac{2n-1}{2n+1} + B_1 \cdot \frac{2n}{2}$	...	...	...	...	...	...	...	...	...	...	.....	0	$\left\{-\frac{1}{2} \cdot \frac{2n-1}{2n+1} + B_1 \cdot \frac{2n}{2}\right\}$					
12	−	$^1V_{10} - ^1V_1$	$^2V_{10}$	$^1V_{10} = ^2V_{10}$, $^1V_1 = ^1V_1$	$= n-2\ (=2)$	1	...	...	...	...	...	...	...	...	n − 2							
13	−	$^1V_6 - ^1V_1$	$^2V_6$	$^1V_6 = ^2V_6$, $^1V_1 = ^1V_1$	$= 2n-1$	1	...	...	...	...	2n−1											
14	+	$^1V_1 + ^1V_7$	$^2V_7$	$^1V_1 = ^1V_1$, $^1V_7 = ^2V_7$	$= 2+1 = 3$	1	...	...	...	...	...	3										
15	÷	$^2V_6 \div ^2V_7$	$^2V_8$	$^2V_6 = ^2V_6$, $^2V_7 = ^2V_7$	$= \frac{2n-1}{3}$	...	...	...	...	...	2n−1	3	$\frac{2n-1}{3}$									
16	×	$^1V_8 \times ^3V_{11}$	$^4V_{11}$	$^1V_8 = ^0V_8$, $^3V_{11} = ^4V_{11}$	$= \frac{2n}{2} \cdot \frac{2n-1}{3}$	...	...	...	...	...	...	...	0	...	...	$\frac{2n}{2} \cdot \frac{2n-1}{3}$						
17	−	$^2V_6 - ^1V_1$	$^3V_6$	$^2V_6 = ^3V_6$, $^1V_1 = ^1V_1$	$= 2n-2$	1	...	...	...	...	2n−2											
18	+	$^1V_1 + ^2V_7$	$^3V_7$	$^2V_7 = ^3V_7$, $^1V_1 = ^1V_1$	$= 3+1 = 4$	1	...	...	...	...	...	4										
19	÷	$^3V_6 \div ^3V_7$	$^1V_9$	$^3V_6 = ^3V_6$, $^3V_7 = ^3V_7$	$= \frac{2n-2}{4}$	...	...	...	...	...	2n−2	4	...	$\frac{2n-2}{4}$	...	$\left\{\frac{2n}{2} \cdot \frac{2n-1}{3} \cdot \frac{2n-2}{3} = A_2\right\}$							
20	×	$^1V_9 \times ^4V_{11}$	$^5V_{11}$	$^1V_9 = ^0V_9$, $^4V_{11} = ^0V_{11}$	$= \frac{2n}{2} \cdot \frac{2n-1}{3} \cdot \frac{2n-2}{4} = A_3$	...	...	...	...	...	...	...	...	0								
21	×	$^1V_{22} \times ^5V_{11}$	$^0V_{12}$	$^1V_{22} = ^1V_{22}$, $^0V_{12} = ^5V_{12}$	$= B_3 \cdot \frac{2n}{2} \cdot \frac{2n-1}{3} \cdot \frac{2n-2}{3} = B_3 A_3$	...	...	...	...	...	...	...	...	...	...	0	$B_3 A_3$	.....	.....	B_3		
22	+	$^2V_{12} + ^2V_{13}$	$^3V_{13}$	$^2V_{12} = ^0V_{12}$, $^2V_{13} = ^3V_{13}$	$= A_0 + B_1 A_1 + B_3 A_3$	...	...	...	...	...	...	...	...	...	...	.....	0	$\{A_3 + B_1 A_1 + B_3 A_3\}$				
23	−	$^2V_{10} - ^1V_1$	$^3V_{10}$	$^2V_{10} = ^3V_{10}$, $^1V_1 = ^1V_1$	$= n-3\ (=1)$	1	...	...	...	...	...	...	...	...	n − 3							

Here follows a repetition of Operations thirteen to twenty-three.

24	+	$^4V_{13} + ^0V_{24}$	$^1V_{24}$	$^4V_{13} = ^0V_{13}$, $^0V_{24} = ^1V_{24}$	$= B_7$	...	...	...	...	...	...	...	...	...	...	.....	.....	.....	.....	.....	.....	B_7
25	+	$^1V_1 + ^1V_3$	$^1V_3$	$^1V_1 = ^1V_1$, $^1V_3 = ^1V_3$, $^4V_6 = ^0V_6$, $^4V_7 = ^0V_7$	$= n+1 = 4+1 = 5$ by a Variable-card. by a Variable card.	1	...	n+1	...	...	0	0	...	...	...	.....	.....	.....	.....	.....	.....	

Portrait: A. E. Chalon, c. 1840. Note G: Lovelace, 1843

Levels of programming languages

Universität Konstanz

Type	Examples	What it is	The same computation
High-level	Python, R, Java, C	• Written for humans, portable • One line becomes many machine instructions	`area = b * h / 2`

Low-level	Assembly	• Readable names for machine instructions. Machine dependent.	`LOAD r1,b`
			`LOAD r2,h`
			`MUL r1,r2`
			`DIV r1,#2`

	Machine code	• Binary instructions the CPU executes	`0001001001000101`
			`0010010011101100`
			`10101101001...`

Interpreter vs. compiler

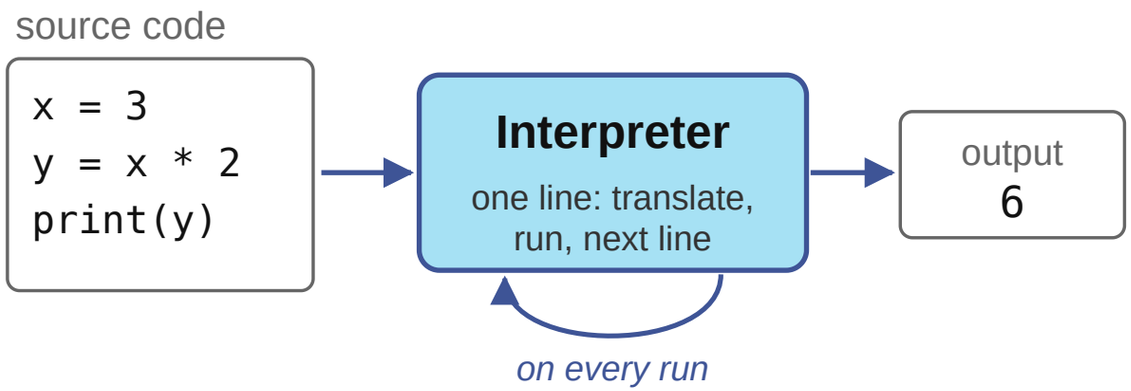
Universität Konstanz

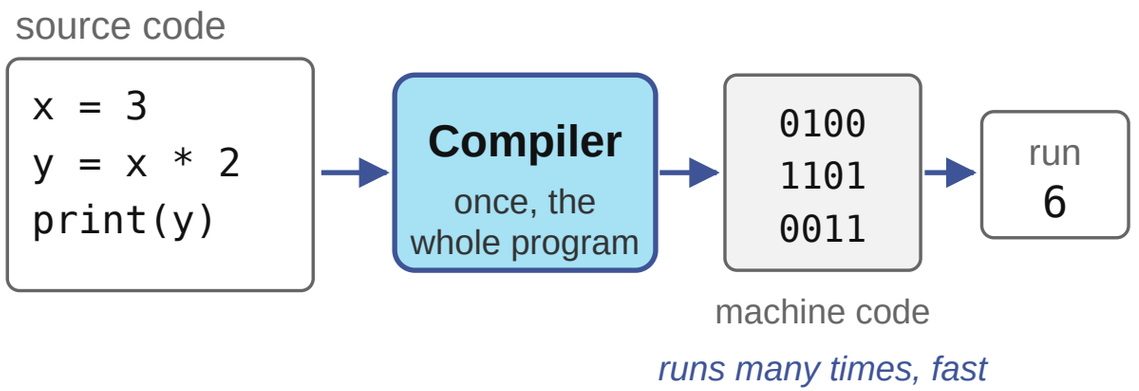
Interpreter: Python, R, JavaScript

- Translates and runs the code **one line at a time**, on every run
- Errors appear only when the line runs, never before

Compiler: C, C++, (Java)

- Translates the **whole program** once into machine code binary file
- Some errors appear before it runs, others require a debugger





Programming paradigms

Universität Konstanz

Paradigms classify languages and programs by **how** they express a computation

Imperative

Statements that change the program's state step by step. Determines **how** to compute.

Declarative

Describes **what** the result should be, not how to compute it.
(for example SQL, session 11)

Object-oriented

Groups data and the instructions that operate on it into **objects**. Models what we compute on.

Functional

The result is the value of functions applied to functions
(based on recursion, session 6)

Python supports all of them: which one you use depends on how you write.

Imperative programming

- The program has a **state** composed of the current values of its variables
- **Assignments** change the state
- **Control structures** choose what runs next, for example conditionals and loops
- **Functions** name a block of steps that can be reused and centrally changed

```
1  total = 0                  # assignment
2  for age in [34, 41]:       # loop
3      if age >= 18:          # condition
4          total += age
5
6  def mean(xs):              # function
7      return sum(xs) / len(xs)
```

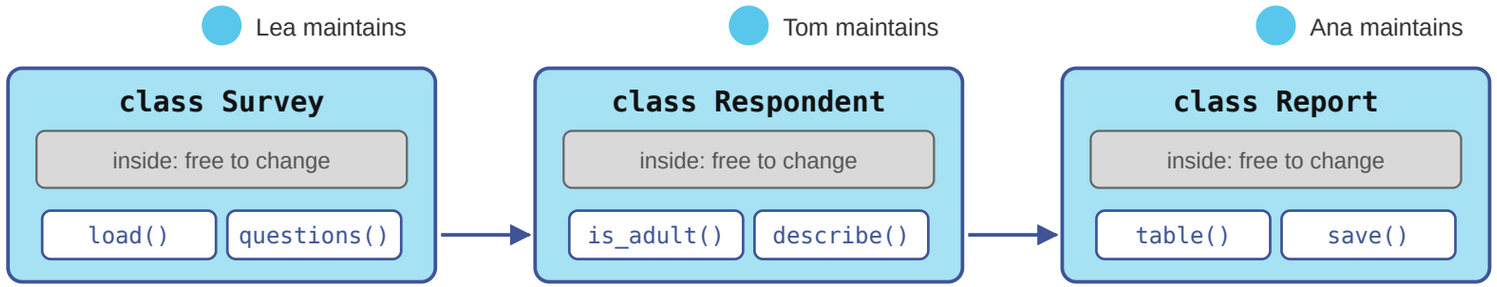
Etymology

imperare, Latin: **to order, to command**. An imperative program is a sequence of orders, carried out one after the other.

Why do we use Object-Oriented Programming?

Universität Konstanz

- Large programs are written by **teams** and **maintained** for years
- The main motivation is code management: OOP gives each part of the program **one responsibility**, in one class, owned by one person or team
- Others use a class only through its **methods**. Its code can change without affecting the other developers. Developers just need to know the object model, not the code inside.



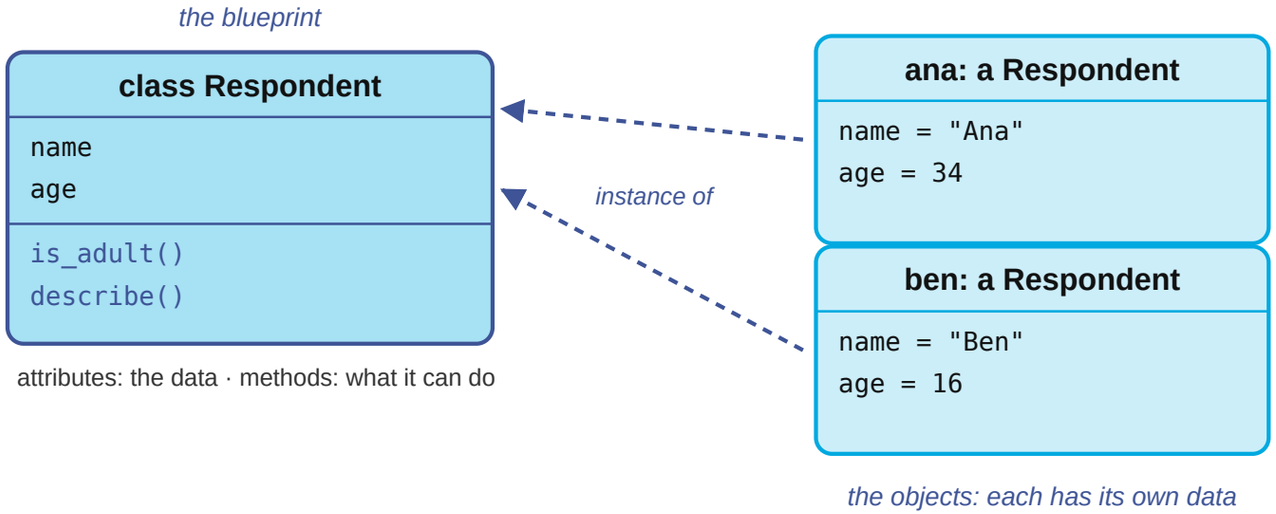
the classes talk only through their methods

A note on scale

For a one-person script, OOP adds little. But it has become a conceptual building block of many programming languages, Python included: every value in Python is an object.

Object-oriented programming

- An **object** bundles **data** (its *attributes*) with the **operations** on that data (its *methods*)
- A **class** is the blueprint. Every object is an **instance** of a class
- You query or change an object by calling its methods: `object.method(arguments)`




```
1  ana = Respondent("Ana", 34)
2  ben = Respondent("Ben", 16)
3  ana.describe()   # 'Ana, 34 years'
4  ben.is_adult()   # False
```

You already use objects

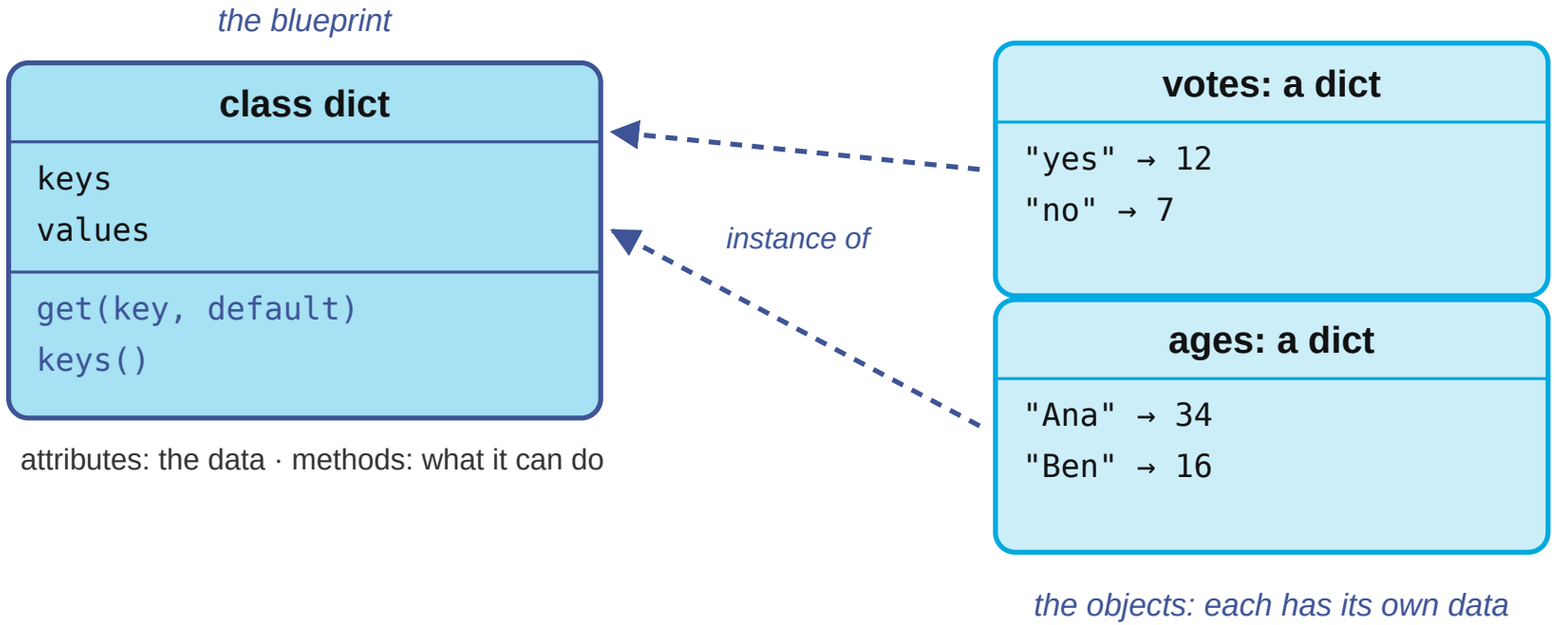
Universität Konstanz

Strings

```
1  s = "Konstanz"
2  s.upper()     # 'KONSTANZ'
3  s.count("n")  # 2
```

Dictionaries

```
1  votes = {"yes": 12, "no": 7}
2  votes.get("no")         # 7
3  votes.get("maybe", 0)   # 0
4  list(votes.keys())      # ['yes', 'no']
```



`s` is a `str` object, `votes` a `dict` object; lists and data frames work the same way.

Your own class and objects

Universität Konstanz

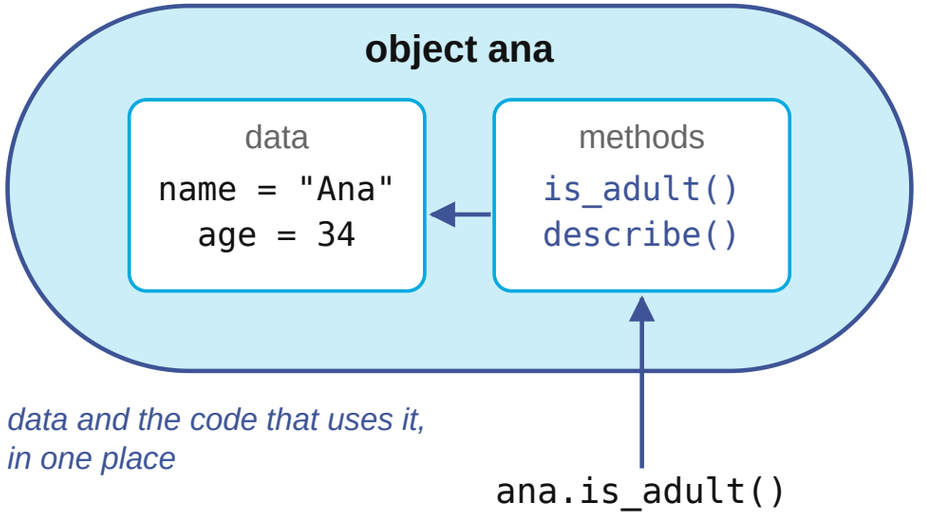
```
 1 class Respondent:
 2     def __init__(self, name, age):       # runs when an object is created
 3         self.name = name                 # attributes
 4         self.age = age
 5
 6     def is_adult(self):                  # methods; self is the object
 7         return self.age >= 18
 8
 9     def describe(self):
10         return f"{self.name}, {self.age} years"
```

```
1 ana = Respondent("Ana", 34)
2 ben = Respondent("Ben", 16)
3 ana.describe()
4 ben.is_adult()
```

```
'Ana, 34 years'
False
```

OOP concepts

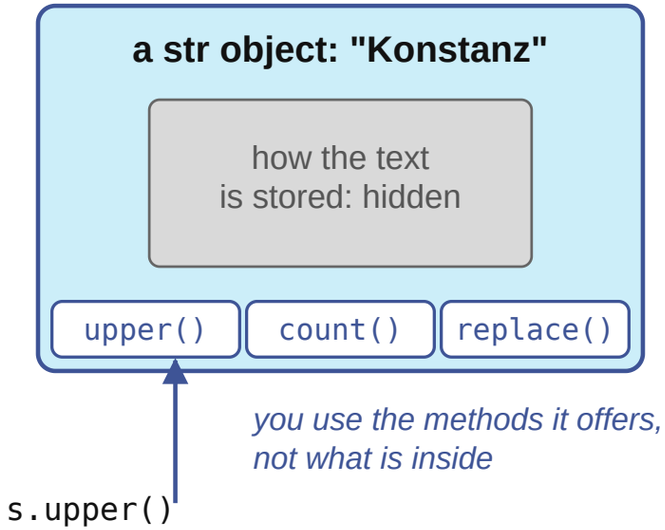
- **Encapsulation:** data and the methods that use it live together in one object.




```
1 ana = Respondent("Ana", 34)
2 ana.age           # 34
3 ana.is_adult()    # True
```

OOP concepts

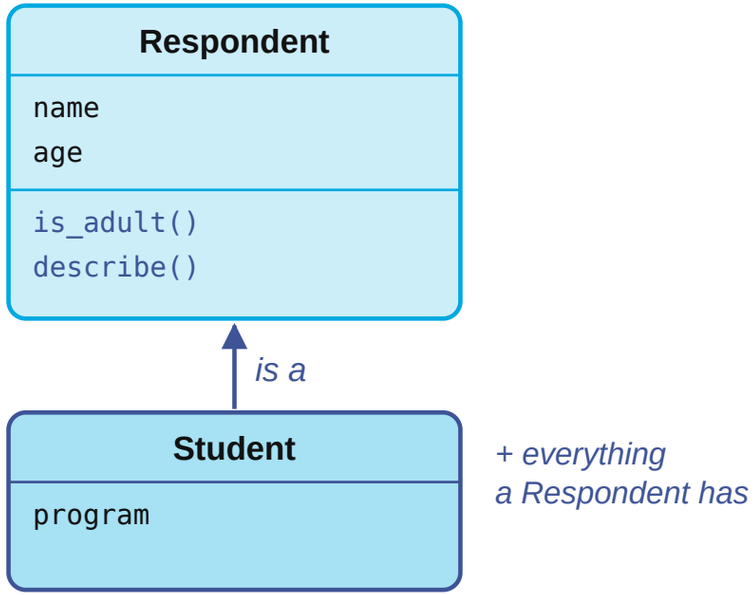
- Encapsulation: data and the methods that use it live together in one object
- **Abstraction:** you use an object through its methods, without knowing how it works inside




```
1  s = "Konstanz"
2  s.upper()   # 'KONSTANZ', but how?
```

OOP concepts

- Encapsulation: data and the methods that use it live together in one object
- Abstraction: you use an object through its methods, without knowing how it works inside
- **Inheritance:** a new class extends an existing one and reuses its methods: a student *is a* respondent

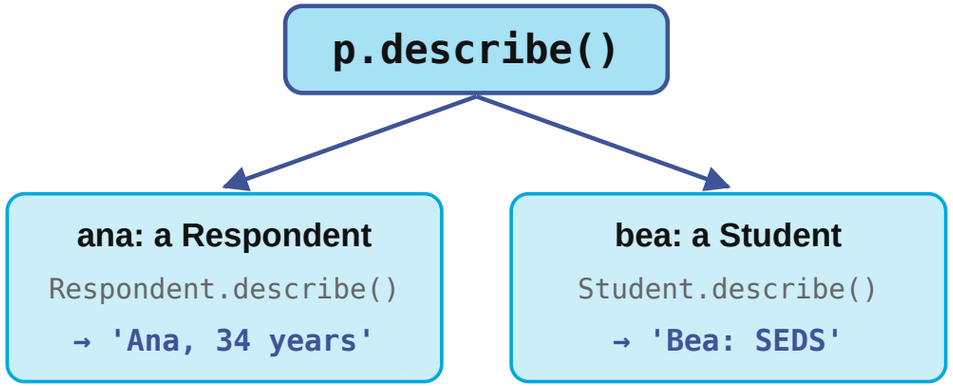



```python
class Student(Respondent):
    def __init__(self, name, age, program):
        super().__init__(name, age)
        self.program = program

bea = Student("Bea", 21, "SEDS")
bea.is_adult()  # True: inherited
```

OOP concepts

- Encapsulation: data and the methods that use it live together in one object
- Abstraction: you use an object through its methods, without knowing how it works inside
- Inheritance: a new class extends an existing one and reuses its methods: a student *is a* respondent
- **Polymorphism:** the same call does the right thing for each class



the same call; each object runs the method of its own class

```
1 class Student(Respondent):
2     def describe(self):
3         return f"{self.name}: {self.program}"
4 ana.describe()  # 'Ana, 34 years'
5 bea.describe()  # 'Bea: SEDS'
```

Software engineering

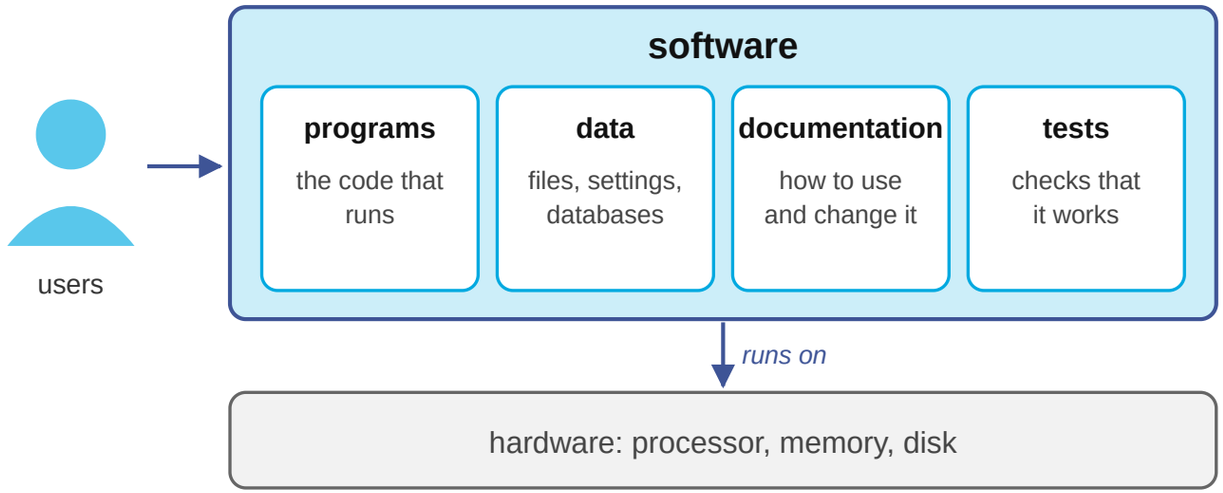
1. Programming in a nutshell
2. **Software engineering**
3. Requirements, specifications, and design
4. Testing

What is software?

Universität Konstanz

Definition: Software

Computer programs, procedures, and possibly associated documentation and data pertaining to the operation of a computer system.



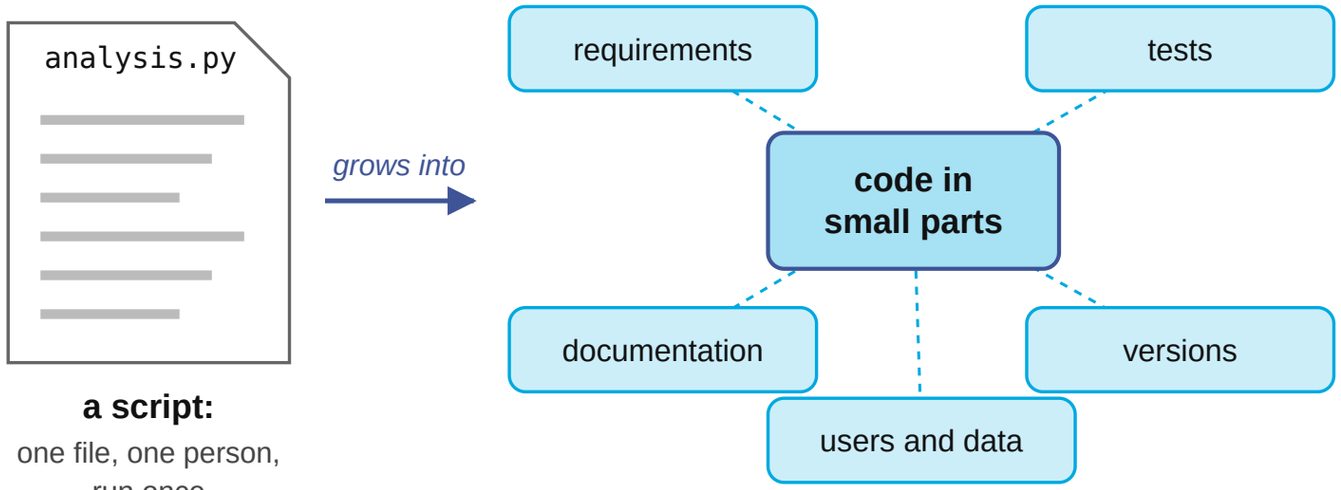
IEEE Std 610.12-1990, *Standard Glossary of Software Engineering Terminology*

From a script to software

Universität Konstanz

Definition: Software engineering

The application of a systematic, disciplined approach to the development, operation and maintenance of software.

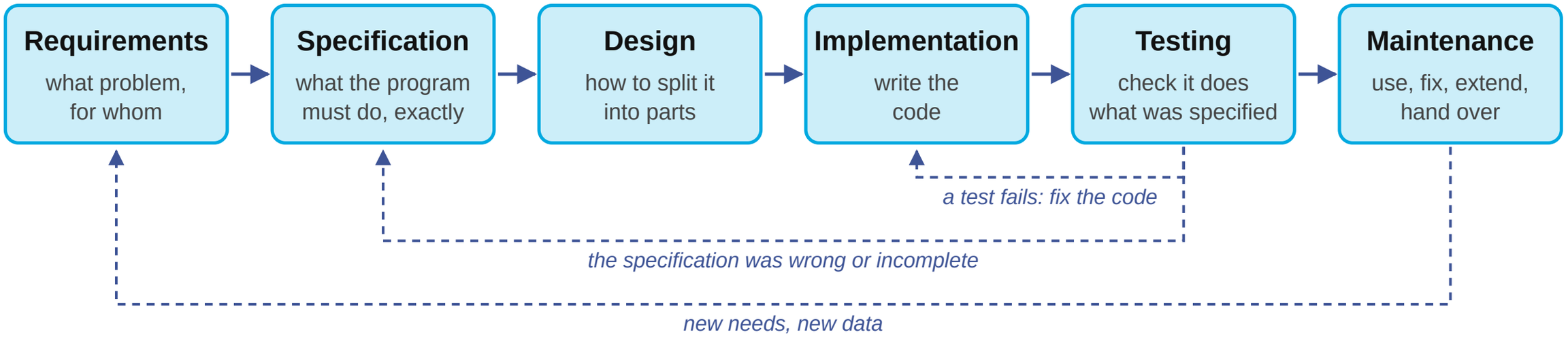


a script:
one file, one person, run once

software: many parts, many people, many years

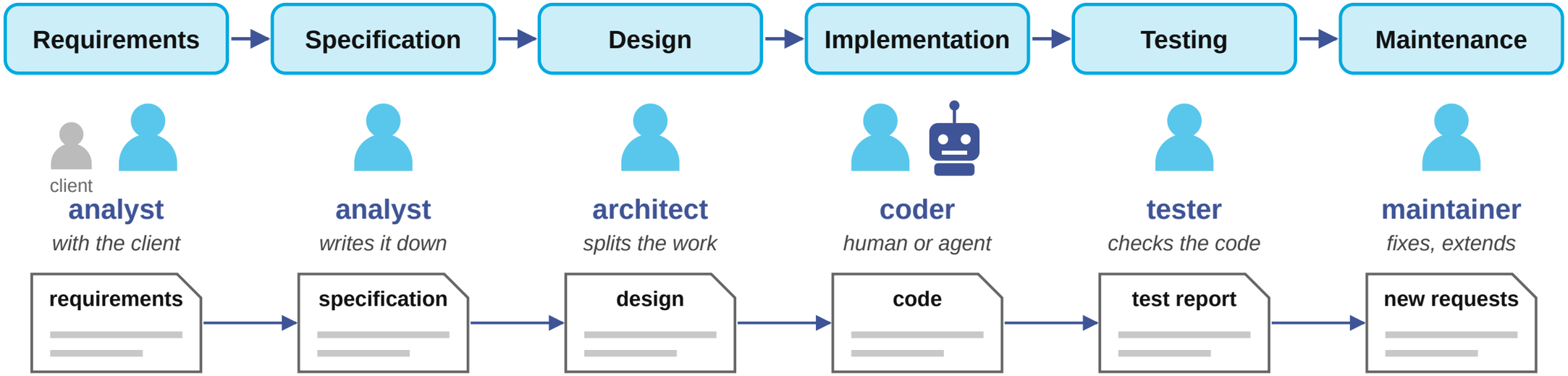
Research code needs this too. Software outlives the paper and others build on it.

A typical software engineering flow



- Real projects go around this loop many times, in small steps (**iterative** development)
- Two steps define correctness: the **specification** says what is right, **testing** checks it

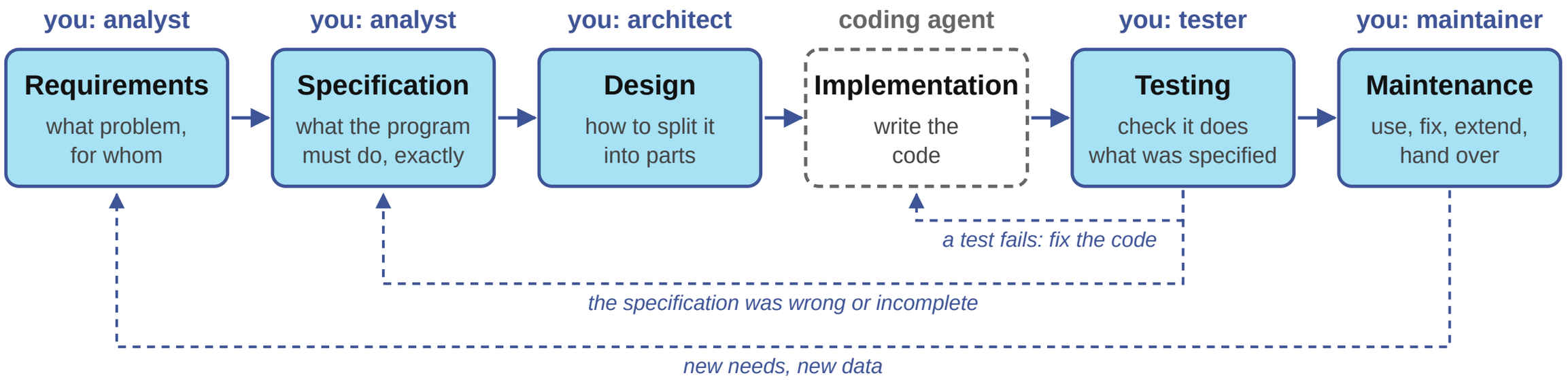
Roles in a software team



each role hands its result to the next one; in a small project, one person plays several roles

- Each stage has an owner: the **analyst** finds out what is needed, the **architect** splits the work, the **coder** implements, the **tester** checks, the **maintainer** keeps it running
- The **coder** can be a person or a coding agent: both work from the same instructions
- In academic research you often play every role yourself but in industry you will have to work with more people

Vibe coding done right



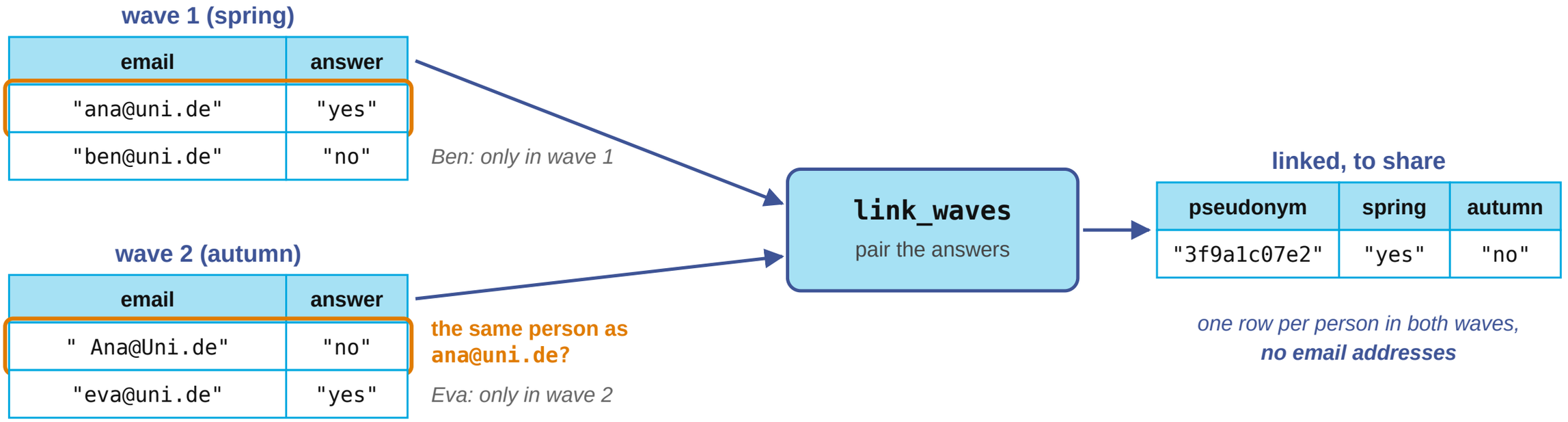
- **Vibe coding**: describe what you want, accept the generated code without reading it, and see whether it seems to work
- A coding agent can take the **coder's** role. Agents can sometimes help on the design and the testing but especially requirements and specifications are dangerous to vibe code

Requirements, specifications, and design

1. Programming in a nutshell
2. Software engineering
3. **Requirements, specifications, and design**
4. Testing

Our example: a panel survey in two waves

Ana, the client, says: *“We asked the same people the same question in spring and in autumn. I need everyone’s two answers side by side to see who changed their mind. Please, no email addresses in what I share with the team.”*



Here we use this request as an example that goes over each stage of the sofware engineering flow, from requirements to maintenance

What are requirements?

Universität Konstanz

Definition: Requirements

Description of what the software must achieve, for whom, and under which constraints and conditions. This is written in the words of the people who need it.

- The **analyst** collects requirements from the **client**, who may be a research team, a colleague, a target user, or yourself in three months
- They describe the need, not the program. *“Everyone’s two answers side by side, with no email addresses”* is a requirement
- Four questions find most of them. **Who** uses it? With **which data**? What must **never** happen? When is it **done**?
- A misunderstanding found here costs a conversation with the client. Found after the code is written, it costs work and risks that can be avoided.

Functional and non-functional requirements

Universität Konstanz

Functional

Functional requirements map inputs to outputs.

In the example:

- Only people in both waves are paired, also if they typed their email differently.
- People only in one wave do not appear in the output
- An empty wave raises an error

Non-functional

How the program must behave beyond its outputs

In the example:

- **security**: users see only their own data
- **privacy**: no email addresses in the output
- **safety**: never delete or change the original data
- performance, reproducibility

Tea met its functional requirements because users could talk with each other as intended. Nobody had written down the non-functional requirements, especially not the one about who is allowed to access user pictures.

What is a specification?

Universität Konstanz

Definition: Specification

A precise description of **what** a program must do, without saying **how**.

- Essentially a better-written version of a requirement.
- The **inputs**, and what may be assumed about them (the *preconditions*)
- The **outputs**, and what is guaranteed about them (the *postconditions*)
- What happens in **edge cases and errors** and what the program must **never** do
- The analyst writes it for two readers. The **coder** implements it, and the **tester** checks the code against it

In this course, you find specifications for each algorithm next to `Inputs:` and `Result:`

From ambiguous request to specification

Universität Konstanz

> *“Write a function that links the two waves of our panel survey, so we can see who changed their answer.”*

A careful coder asks these five questions. A hurried coder, human or agent, answers them alone and in silence.

1. When are two records the same person? Is `" Ana@Uni.de"` the same as `"ana@uni.de"`?
2. What if someone answered twice in the same wave?
3. What happens to people who answered only once?
4. May email addresses appear in the result?
5. May the function change the lists it was given?

Writing the specification

```
 1 def link_waves(wave1, wave2):
 2     """Pair the answers of the people in both waves.
 3
 4     wave1, wave2: lists of (email, answer) pairs.
 5     Emails match ignoring case and surrounding spaces.
 6     If someone answered twice in a wave, the last answer counts.
 7     Returns: a dict from pseudonym(email) to (answer1, answer2),
 8       only for the people in both waves.
 9     Raises: ValueError if a wave is empty.
10     Never: puts an email in the result or changes the lists.
11     """
```

- Each line answers one of the five questions
- One extra line settles the empty case
- Nothing here says **how** to link the waves, only **what** we want

The specification is the coder's instructions

Universität Konstanz

Vague instructions

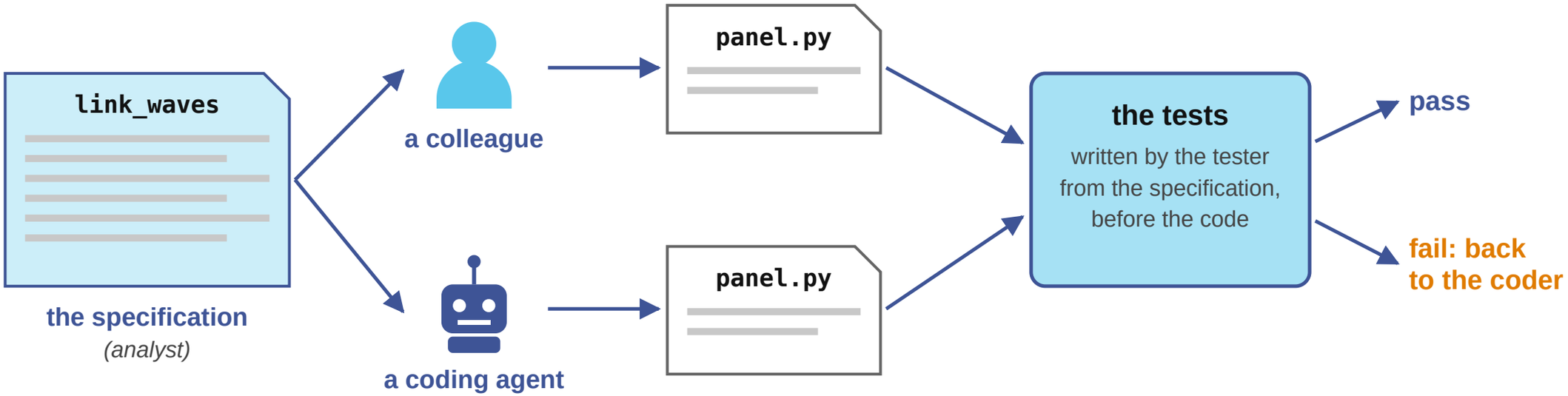
"Write a function that links the two waves."

Specified instructions

"Write `link_waves(wave1, wave2)`. Each wave is a list of (email, answer) pairs. Match emails ignoring case and spaces. If someone answered twice, keep the last answer. Return a dict from `pseudonym(email)` to the two answers, for the people in both waves. Raise `ValueError` if a wave is empty. Never put an email in the result or change the lists. Use only the standard library."

- Precise instructions get better code from any coder.
- **Important:** specifications give you something to **test** the code against
- A rule that must never be broken needs more than instructions. Enforce it with permissions, backups, and separate test and production data

One specification, any coder



- A colleague and a coding agent receive **the same instructions** and their code has to pass **the same tests**
- The tester writes the tests from the specification, not from the code. Tests are done against was asked rather than what was coded.
- Ideally, coders and testers are different people (and tend to hate each other)

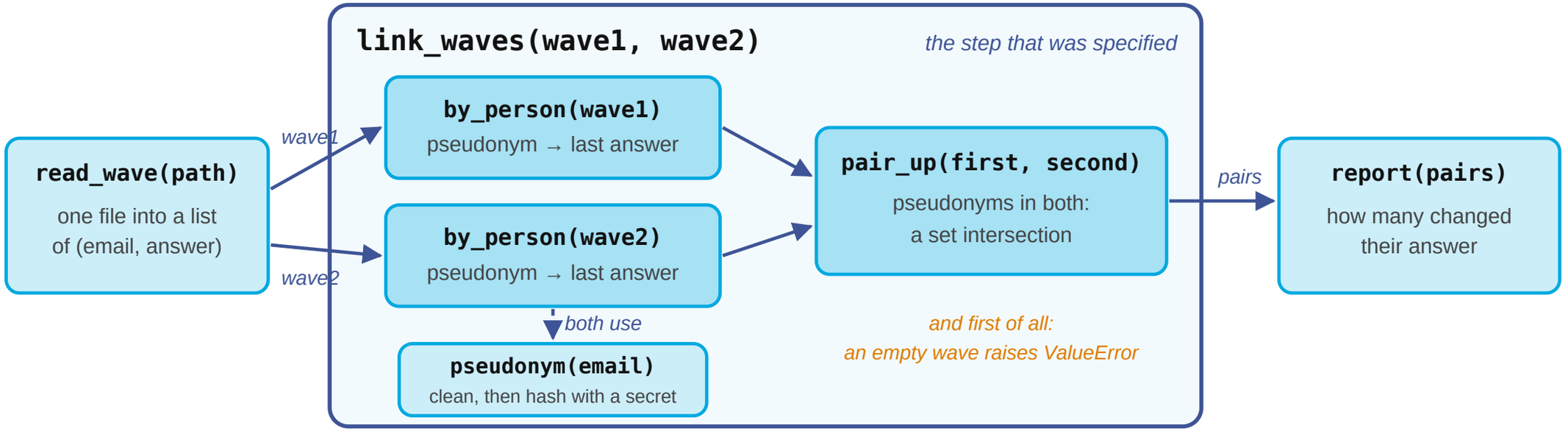
What, not how



- Requirements and the specification say **what** the software must do. The design says **how**
- Clients and analysts describe the need and the result, not the solution. *“Everyone’s two answers side by side”* leaves the how open
- The how decides the **cost**. Two requests that sound alike can differ by years of work, as in the comic
- Estimates of cost and effort are made at the **design** stage, once the architect knows how the work will be split

xkcd 1425, “Tasks”, by Randall Munroe (2014), CC BY-NC 2.5

Design: Small subtasks with clear connections



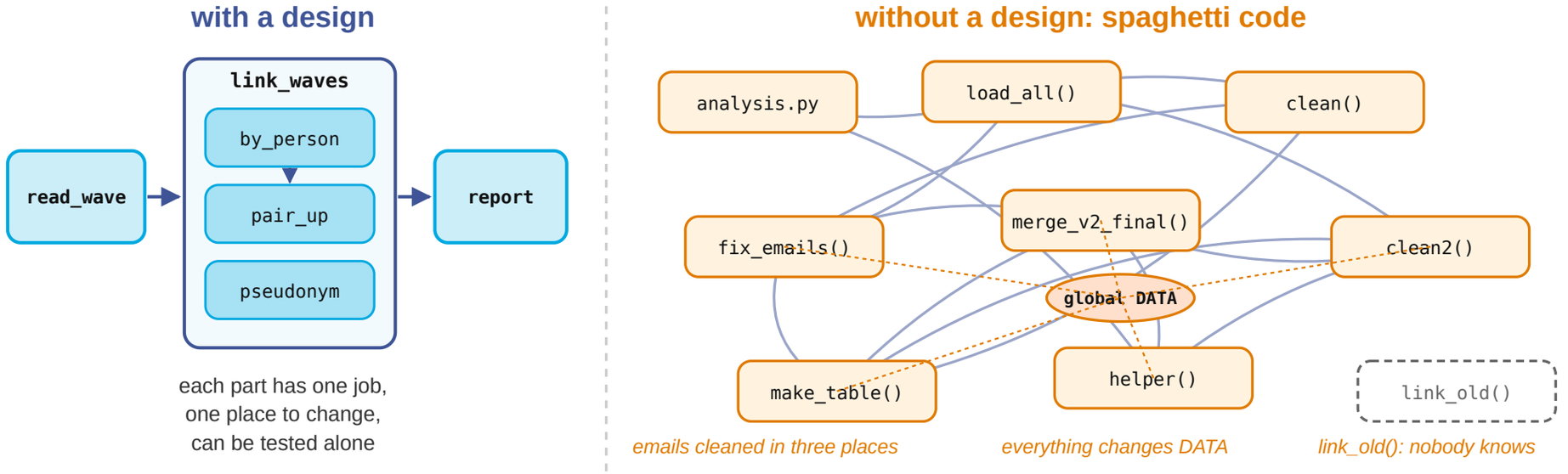
- The **architect** designs how to implement the specifications by breaking the program into **small parts**, each with one job and its own specification
- The architect also decides **what data flows between the parts** and in which data structure
- Small parts are easier to test, easier to replace, and easier to hand to a coder. They are usually the size of a function or a kind of atomic component

Design: choosing the data structures

The program needs to…	Data structure (session 3)	Example
keep items in order	list	the (email, answer) pairs of a wave
look something up by a key	dictionary	pseudonym → answer, one per wave
check membership fast	set	the pseudonyms in both waves
serve first come, first served	queue	a waiting list for a seminar (to be covered in session 5)

A coder, human or agent, writes better code inside a structure the architect chose than inside one it invented on the spot

Skipping the design: spaghetti code



- Without a design, every new need is patched in wherever it fits: duplicated parts, a global variable that everything changes, code nobody dares to remove
- Every later change then touches many parts, and no part can be tested on its own
- Design is **pre-work**: an hour of design saves days of coding, debugging and testing down the road

Testing

1. Programming in a nutshell
2. Software engineering
3. Requirements, specifications, and design
4. **Testing**

What is a test?

Universität Konstanz

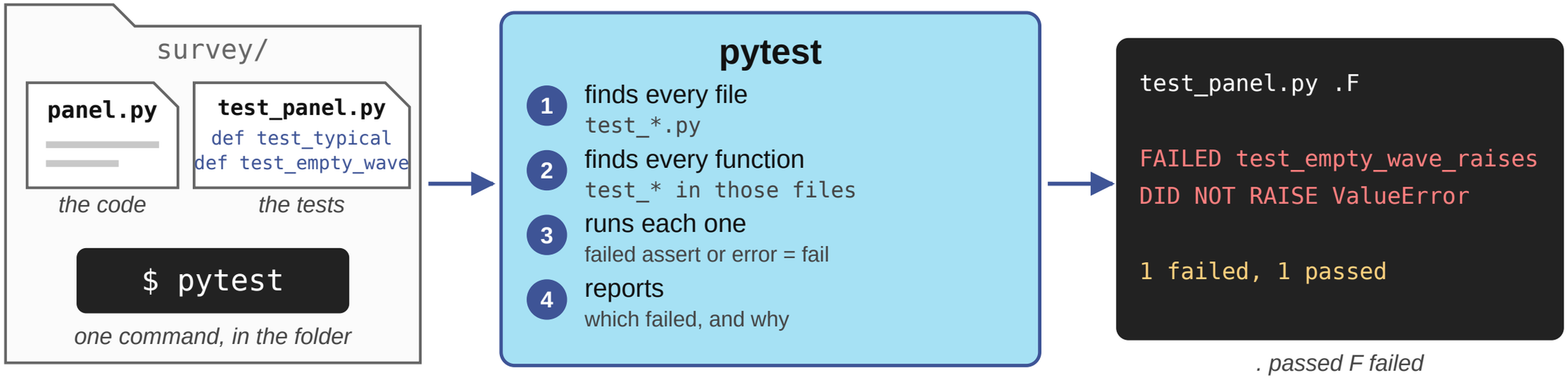
Definition: Test

A small program that runs your code on an input whose correct output you know, and checks that the code produces the expected output.

```
1 w1 = [("ana@uni.de", "yes"), ("ben@uni.de", "no")]
2 w2 = [("ana@uni.de", "no"), ("eva@uni.de", "yes")]
3 assert list(link_waves(w1, w2).values()) == [("yes", "no")]
4 assert "ana@uni.de" not in str(link_waves(w1, w2))
```

- The specification says what should happen. The **tester's** tests check that it does
- A failing `assert` stops the program with an error. This makes errors loud and prevent quiet errors that are only detected later

What is pytest?



- **pytest** is a Python package for running tests
- You can install it with `pip install pytest`
- A test is a function named `test_...` in a file named `test_...`. It **passes** if it runs to the end and **fails** on a failed `assert` or an unexpected error
- One command finds all the tests, runs them, and reports which ones failed and why

Tests with pytest: the test file

```
 1 # test_panel.py
 2 import pytest
 3 from panel import link_waves
 4
 5 W1 = [("ana@uni.de", "yes"),
 6       ("ben@uni.de", "no")]
 7 W2 = [("ana@uni.de", "no"),
 8       ("eva@uni.de", "yes")]
 9
10 def test_typical():
11     pairs = link_waves(W1, W2)
12     assert list(pairs.values()) == [("yes", "no")]
```

- The test file imports the function it tests from `panel.py`
- `W1` and `W2` are small test data, shared by all tests
- `test_typical` checks the ordinary case. Ana answered in both waves, so there is one pair

Tests with pytest: edge cases


```python
def test_case_and_spaces_are_ignored():
    w2 = [(" Ana@Uni.de", "no")]
    assert len(link_waves(W1, w2)) == 1

def test_no_emails_in_result():
    pairs = link_waves(W1, W2)
    assert "ana@uni.de" not in str(pairs)

def test_empty_wave_raises():
    with pytest.raises(ValueError):
        link_waves(W1, [])
```

Each test checks one line of the specification. The last one uses `pytest.raises` to check that an error is raised

Example: A quick implementation meets the tests

```
1 def link_waves(wave1, wave2):
2     first = dict(wave1)
3     return {email: (first[email], answer)
4             for email, answer in wave2
5             if email in first}
```

A coder’s first try. Plausible, short, and wrong three times

```
$ pytest -q
.FFF
FAILED test_case_and_spaces_are_ignored
  assert 0 == 1
FAILED test_no_emails_in_result
  'ana@uni.de' is contained here
FAILED test_empty_wave_raises
  DID NOT RAISE ValueError
3 failed, 1 passed
```

- It compares emails exactly, so `" Ana@Uni.de"` becomes a new person and panel members **quietly disappear**
- It keys the result by email, so the privacy requirement is broken without any error
- The fix is to clean each email, key the result by `pseudonym(email)`, and raise an error if a wave is empty. Then all four tests pass

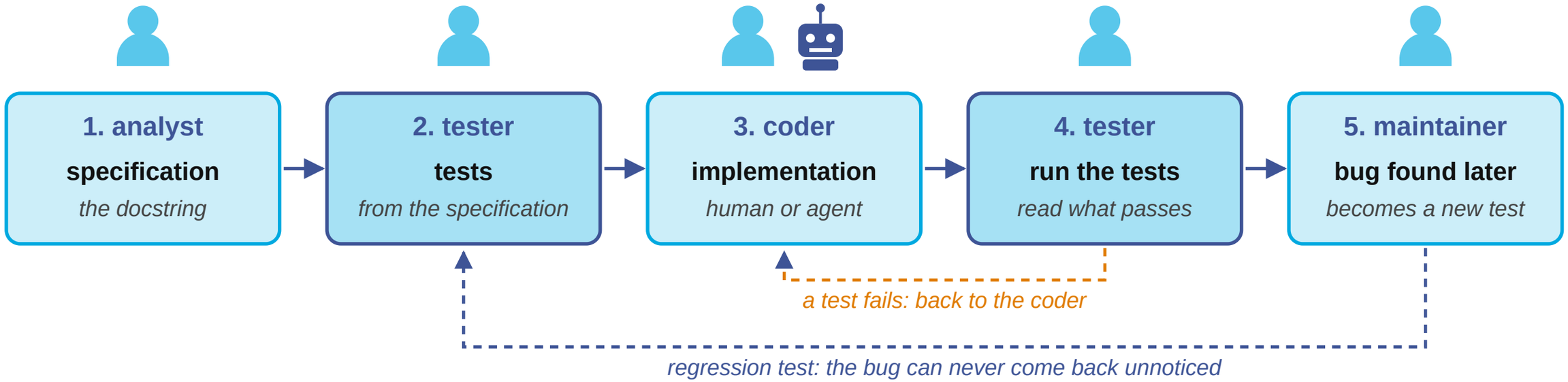
Choosing test cases

Universität Konstanz

Kind	Example input	Expected
Typical	Ana in both waves, Ben and Eva in one	one pair, `("yes", "no")`
Spelling	`" Ana@Uni.de"` and `"ana@uni.de"`	the same person
Duplicates	Ana twice in wave 1	her last answer
Boundary	nobody in both waves	`{}`
Empty	`[]` as a wave	`ValueError`
Must not happen	an email in the result, a changed list	never

Write one test for every line of the specification, and more wherever it draws a line, such as spelling, duplicates, nobody in common, and empty input

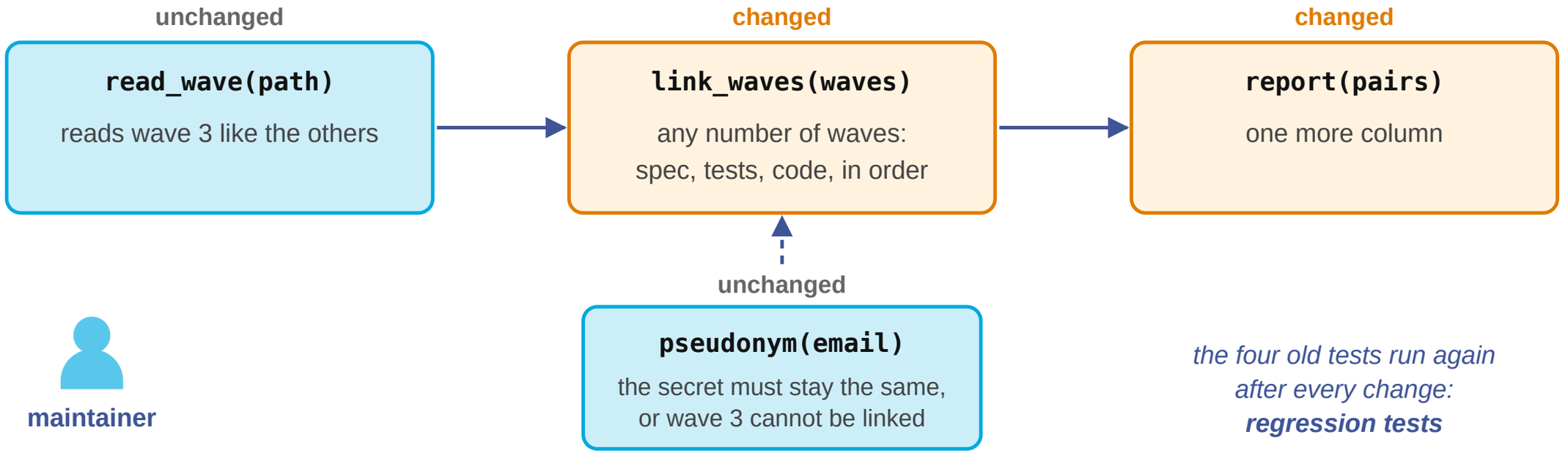
Test first, then implement



- The tests exist **before** the code, so they record what was meant and not what the code happens to do
- The tests are your measurement. They replace the feeling that the code works with evidence that it does. Adapting tests to existing code is a mistake that invalidates the original purpose of testing

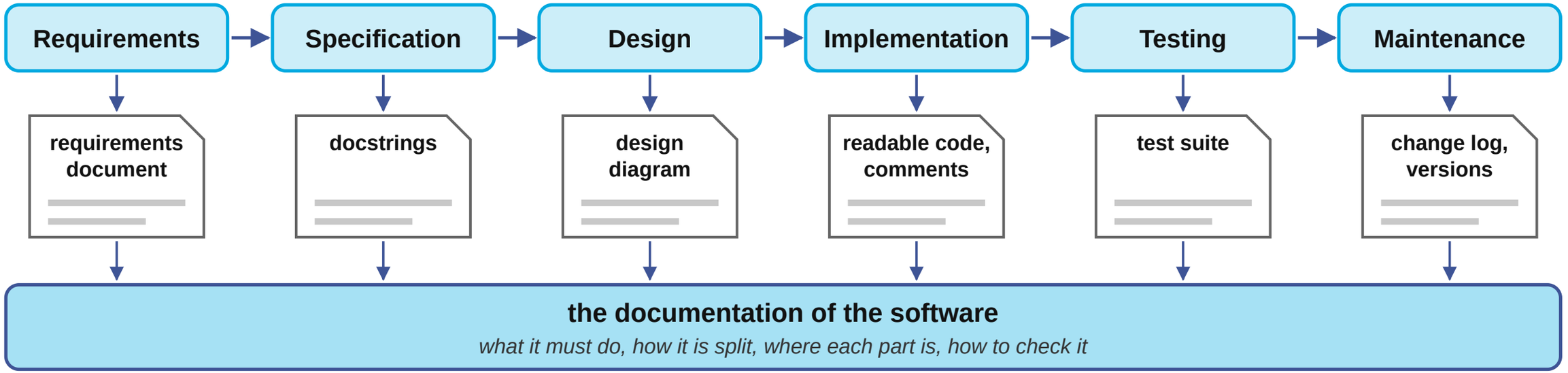
Maintenance example: the third wave arrives

“Great! In spring we run a third wave. Can we link all three?” (Ana, a year later)



The **maintainer** goes through the flow again on a small scale. First the specification changes, then the tests, then the code

The flow leaves documentation behind



Symbian

20 million lines of code and no map of them. Moving an element by 5 mm needs to know which part draws the screen and what depends on it. A design document would have said where; tests would have said what breaks

What tests cannot do

> *“Program testing can be used to show the presence of bugs, but never to show their absence.”*
> Edsger W. Dijkstra, *Notes on Structured Programming* (1970)

- Tests check only the cases you thought of. A missing requirement produces missing tests
- **Software verification** goes further. It *proves*, with mathematics, that a program meets its specification for **every** input. It is expensive, so it is used where failure is unacceptable, as in aircraft control, railway signaling and operating system kernels
- For research code, the practical combination is a specification, tests, and **reading** the code you did not write

A proof instead of a test: a loop invariant

Universität Konstanz

Loop invariant

A statement that is true before every turn of a loop. Here: `total` is the sum of the first `i` elements

```
 1  def mean(xs):
 2      """xs: a non-empty list of numbers.
 3      Returns: their mean."""
 4      total = 0
 5      i = 0
 6      while i < len(xs):
 7          # invariant: total == sum(xs[:i])
 8          total = total + xs[i]
 9          i = i + 1
10      return total / len(xs)
```

1. **Before the loop**: `i` and `total` are 0, the sum of no elements
2. **Each turn**: `total` gains `xs[i]` and `i` grows by 1, so the invariant still holds
3. **After the loop**: `i == len(xs)`, so `total` is the sum of all elements. The list is not empty, so there is no division by zero

A test checks one list. This argument covers **every** non-empty list. Verification tools check such arguments by computer

Summary

Universität Konstanz

- A **program** is a precise list of instructions, run by an **interpreter** or translated by a **compiler**
- **Paradigms** differ in how they express a computation. Imperative programs change state step by step, and **objects** bundle data with methods, as in `s.upper()` or your own classes
- **Software engineering** puts coding inside a loop of requirements, specification, design, implementation, testing and maintenance
- **Requirements** say what is needed, the **specification** says exactly what the program must do, and the **design** splits it into small parts
- A **test** runs the code on cases with known answers. Write the tests first, then implement, whoever the coder is
- With coding agents, implementation is cheap. **Everything around it** is the part you cannot delegate

Research assistant job opening: help review the psychological literature for simulations of human behavior with LLMs. Interested? Drop me an email or talk to me after class

Next session: Algorithms